

I/O rendszer és fájlkezelő

Az előadás anyaga

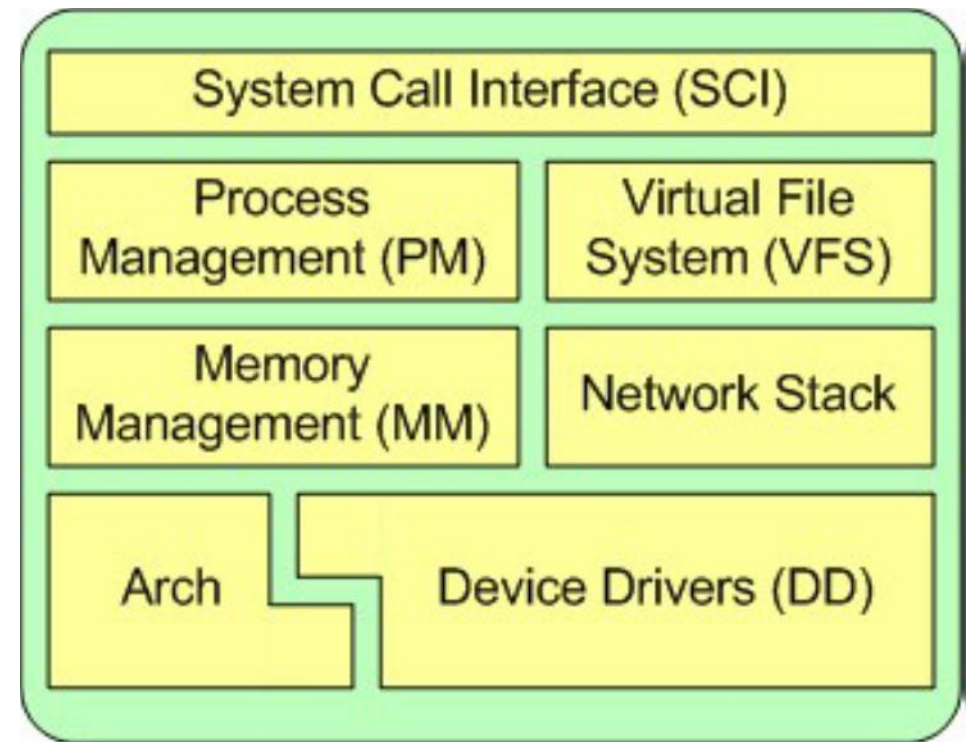
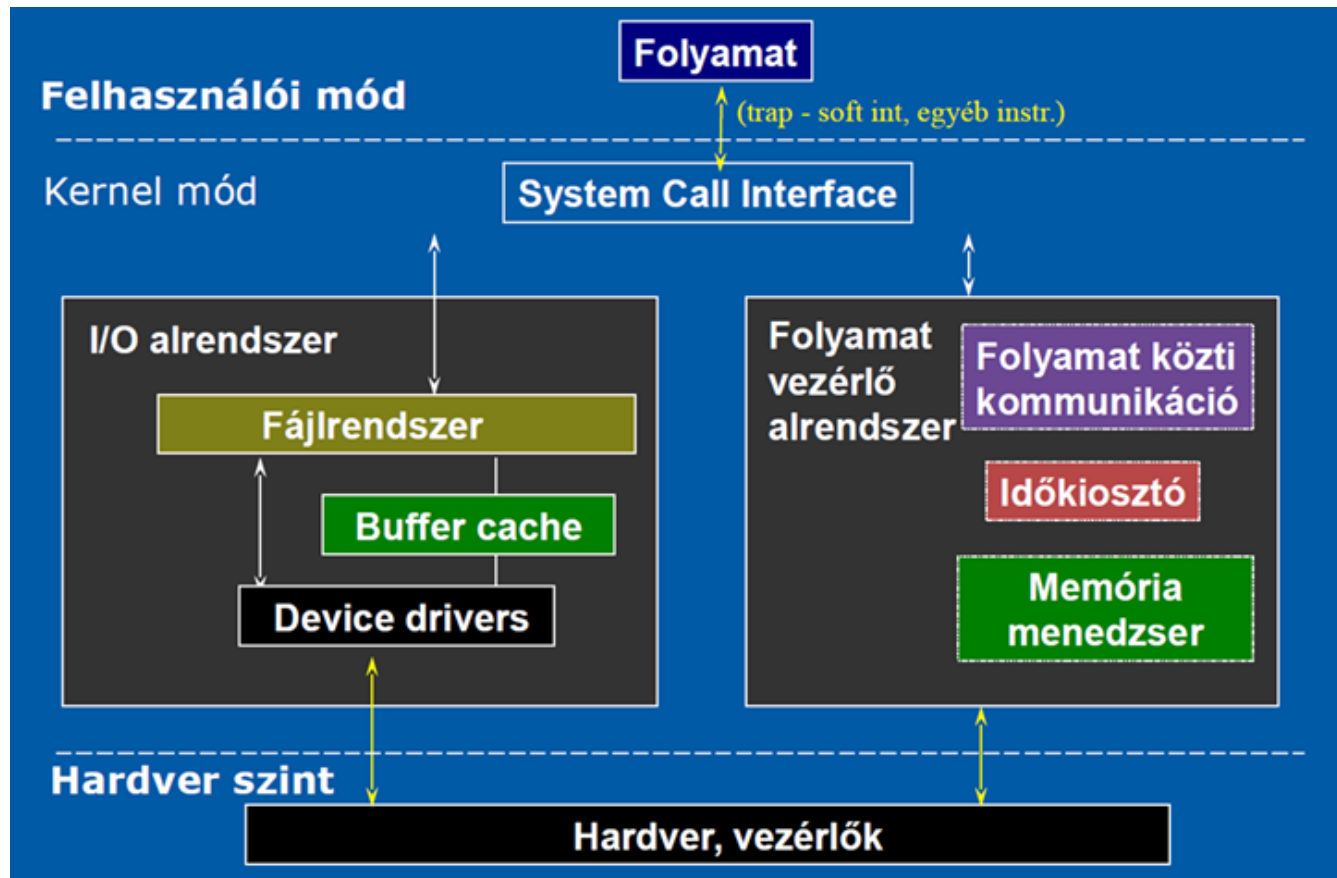
Dr. Vadász Dénes: Operációs rendszerek jegyzet és
a felhasznált irodalom alapján készült el

Témakörök

- Az I/O alrendszer, különböző látásmódokkal
- Eszközök osztályai, eszköz-driverek
- Logikai diszk, partíció fogalom
- Fájlok, szervezettség, elérés
- Fájlrendszer megvalósítási feladatok
 - jegyzékstruktúra megvalósítás,
 - blokk hozzárendelés fájlokhoz,
 - szabad blokk menedzselés.
- Unix fájlrendszer.

UNIX funkcionális felépítése - ismételés

I/O alrendszer



I/O alrendszer - feladatai

„Feladatai:

- *Elrejt*ni az eszközök specialitásait (alkalmazás-futtató, programozók stb.)
- *Kényelmessé tenni* az eszközhasználatot.
- *Menedzselni* az eszközöket, fájlokat.
- Védi azokat, ütemezi, konkurens vagy kizárólagos hozzáféréseket biztosít.

I/O eszközök - eszközök fajtái

Az input-output eszközöket két csoportra bonthatjuk:

A. *Karakteres eszközök.*

B. *Strukturált (blokkos) eszközök.*

I/O eszközök - eszközök fajtái

A. Karakteres eszközök

A karakteres eszközökön olyan eszközöket értünk, amelyek fogadják vagy kibocsátják a karakterek sorozatát.

Ilyen eszközök nem címezhetők és a keresési műveletet sem tudják végrehajtani.

I/O eszközök – karakteres eszközök

A. Karakteres eszközök (pl. billentyűzet, egér, soros port, képernyő, analóg-digitál átalakító (pl. hangkártya) stb.).

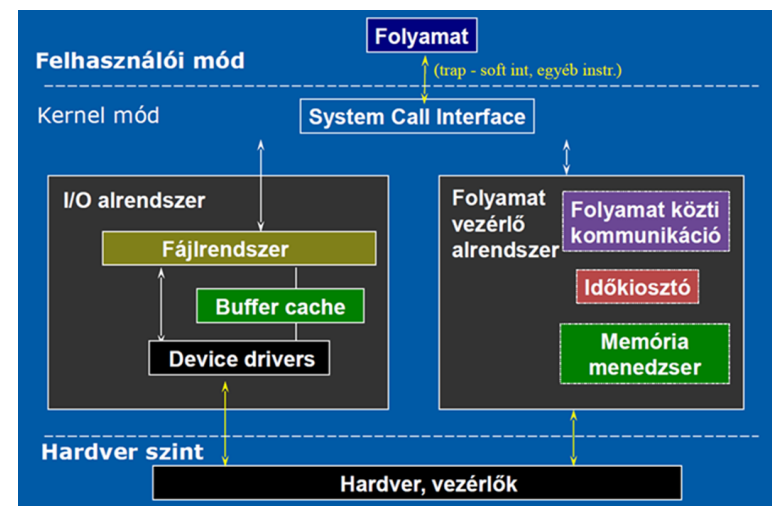
- Strukturálatlanul (pl. karakter-folyamként) kezelhető.
- A driver fölött szűrés lehet (nem mindent ad át felfele)
- *Akár diszk is kezelhető* karakteresen.

I/O eszközök – blokkos eszközök

B. Blokkos eszközök: információt *blokkokban tárolják* és a *blokkok saját címmel rendelkeznek*.

Strukturált (blokkos) eszközök (pl. diszkek).

- *blokkos I/O-val* is kezelhetők,
- fájlrendszer is alakíthatók ki rajtuk,
- a *buffer cache* egy *gyorsítótár* mechanizmuson át kezelhetők.



I/O eszközök - időzítők

Harmadik: olyan eszközök, amelyek egyik nagy csoportba sem sorolhatók be, mivel vagy nem címezhetők blokkonként, ill. nem is fogadnak, vagy nem is generálnak karaktersorozatot.

Ilyen eszközök pl.: az *időzítők*, amik jól meghatározott *időintervallumonként megszakításokat hoznak létre.*

I/O eszközök, kezelése - a felhasználói látásmód

Szimbolikus neven látja az *eszközöket, fájlokat*:

- lát *eszközöket* (köztük logikai eszközöket),
- látja a *fájlok halmazát* (file pool),
- lát *hierarchiát* (jegyzék stb. fogalmak)

A *felhasználói felülettel* (burok, v. GUI) kezeli ezeket (készít, töröl, másol, mozgat, eszközt kezel stb.)

Ismer *tulajdonossági és hozzáférési kategóriákat* (védelem).

I/O eszközök, kezelése - a programozó látásmódja

A process szemszögéből minden *I/O eszköz* – az egy *csatorna* (*stream*).

- Nyitott *csatornákat* (*stream*) lát.

Ezek azonosítója: *fájl-leíró/pointer, fájl-deszkriptor*.

- *Nyithat/zárhat csatornákat* (*open()*, *fopen()*, *close()*, *fclose()*).
- *A csatornákon adatokat mozgat* (*byte/karakter, rekord, blokk stb.*).

I/O eszközök, kezelése - a programozó látásmódja

Ez függ az *elérési módtól*:

- *szekvenciális eléréssel*,
- *direkt eléréssel*.

A csatorna "megszüntethető" a lezárásával: az *explicit close*, *fclose* stb. rendszerhívásokkal.

I/O eszközök, kezelése - a programozó látásmódja

- Az *adatmozgató rendszerhívások*: *read, write, put, get, putchar, getchar, seek* stb.
- Külön *rendszerhívások a jegyzékekre*: *opendir, readdir, mkdir, rmdir* stb.

I/O kezelés - OS látásmódja

Az OS-ek egyik legfontosabb feladata:

- *az összes I/O eszköz vezérlése,*
- *interfész biztosítása,*
- *védelmet és menedzsmentet biztosítása.*

Az operációs rendszer "*látásmódja*" ezért meglehetősen *bonyolult*.

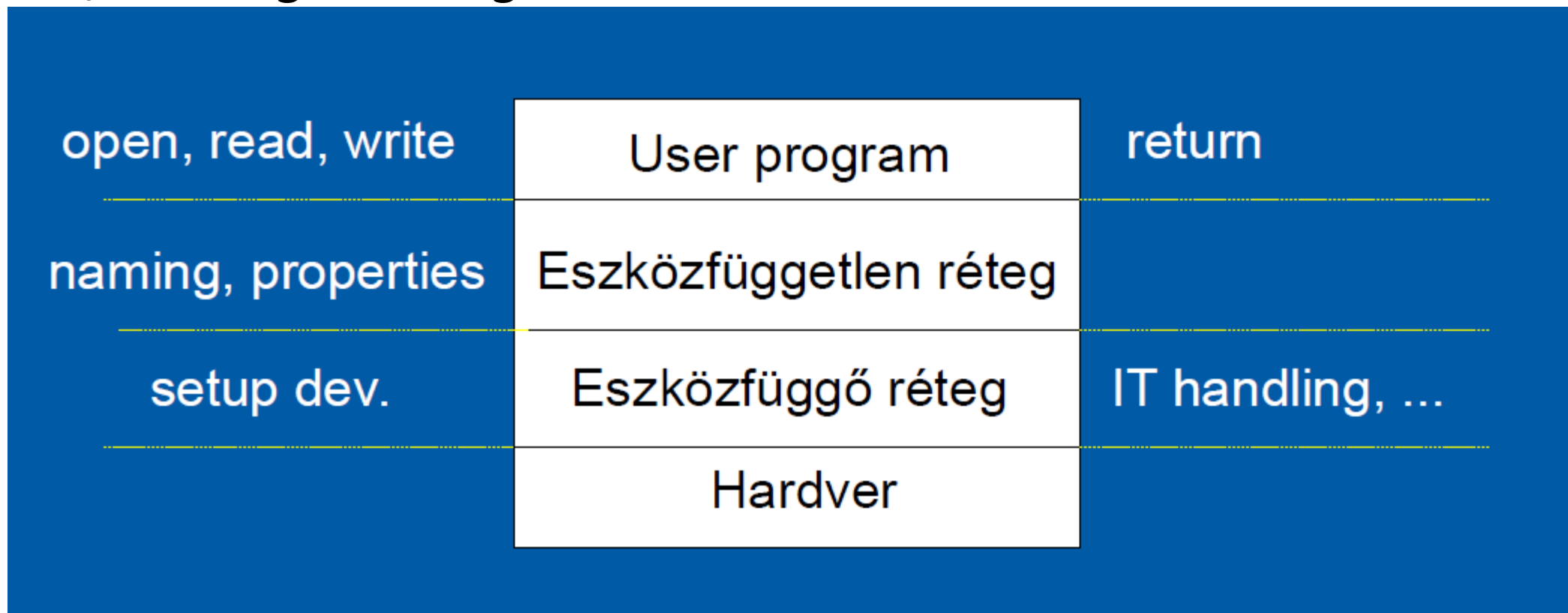
I/O kezelés - OS látásmódja - alapelvek

Az I/O szoftverek rétegekbe szervezése (egy felsőbb réteg magasabb szintű absztrakciót biztosít, az alacsonyabb réteg pedig szolgáltatást a felső réteg számára).

- Biztosítani kell az *eszközfüggetlenséget* (eszköz változtatásnál ne kelljen módosítani a programot))
- *A hibakezelés legyen elosztva* (a hibákat kezeljük minél közelebb a hardver-hez.)

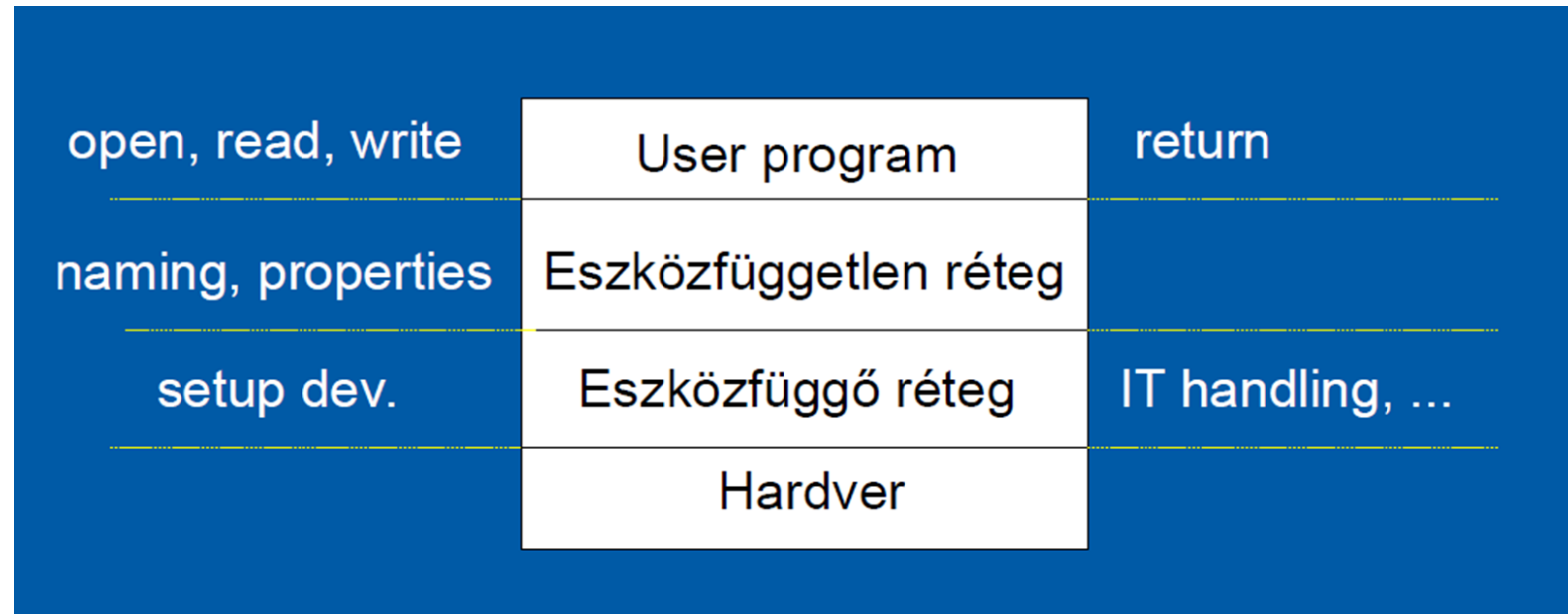
I/O kezelés - Az I/O rétegezettség

Az I/O rétegezettség



I/O kezelés – User szint

Forrás: Vadász



Forrás: Vadász

I/O kezelés - Eszközfüggetlen réteg

Eszközfüggetlen réteg szerepe:

- elrejt az *eszközök közötti különbségeket*,
- *magasabb szintű kezelhetőséget **biztosít** az eszközökhöz,*
- *blokkos eszközökhöz fájlrendszert biztosít.*
- *meghívja a megfelelő eszközkezelőt,*
- *leírókat hoz létre és kezel.*

I/O kezelés - Eszközfüggő réteg

Eszköz meghajtókat (driver-eket) tartalmaz.

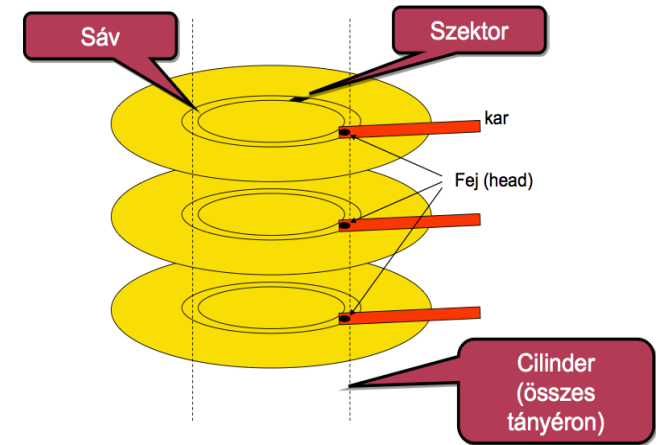
Device driver:

- szerepe egy *eszköz kezelése*,
- *megszólítható: felülről call (hívás), alulról megszakítás*,
- *dinamikus betöltődés*,
- *eszközgyártó készíti.*

Diszkek, blokkos eszközök

Blokknyi egységben történik az adatáramlás.

- Oldal-sáv-(cilinder)-szektor (blokk) fogalmak
 - adattárolási egység *a szektor* (blokk),
 - *oldal-sáv-szektor* címek léteznek.
- Az *oldal-sáv-szektor* címek *leképezhetők logikai blokkcímekre* (ezt maga a kontroller végezheti: *LBA – Logical Block Addressing*): felülről *0-n*-ig számozott blokkok sorának láthatjuk.



Forrás: OPRE jegyzet, BME

Diszk, blokkos eszközök - Logikai diszk, partíció

Logikai diszk, partíció

- Egy *fizikai diszken* **0-n**-ig számozott *blokkok sora* van.
- Mindegyiknek *saját szimbolikus neve* van.
- Mindegyiknek *saját drivere* van.
- Egy fizikai diszken *több partíció lehetséges*.

Megj.: A szektor mérete *512 byte*, az *Advanced Format szabvány* használó egységeknél 4 kilobájt.

A CD-ROM szektorainak hossza 2 kilobájt (2352 bájt).

SSD, blokkos eszközök - Logikai diszk, partíció

SSD esetén *NAND flash-ekre* mentjük az adatokat.

A NAND a “nem-felejtő” memória, nem kell frissíteni, megtartja az állapotát.

A NAND *flash rácsszerkezetes felépítésű* (grid), az elrendezést, *blokknak* hívjuk.

A grid-et alkotó sorok a lapok, ezek mérete 2K, 4K, 8K, 16K lehet. Egy blokkon belül 128 vagy 256 lap van. Egy teljes blokk mérete így tipikusan 256KB és 4MB közé esik.

Diszkek, blokkos eszközök - Logikai diszk, partíció

Egy partíciót kijelölhetünk:

- *Egy OS boot szektornak (erről indítunk),*
- *Kilapozási területnek (virt. mem.),*
- *OS fájlrendszer területének.*

Diszkek, blokkos eszközök - Particionálás

Particionálás

- Részben az *OS-től független dolog!* (BIOS!)
- “Alacsonyabb” *szintű SW-ek kellenek partíciók kezeléséhez* (partíció létrehozásához, törlés, a boot partíció kijelölés, stb.)
- Minden OS-ben megtalálhatók ezek a SW-ek.
- A partíciókat az OS-ek nem egyformán kezelik.

Diszkek, partíciók - kötetek

Partíciók – kötetek (*volume*)

- Partícionálással egy *nagy diszket* *részekre osztunk*.
- Vagy - *kisebb diszkeket* (*partíciókat*) *összevonjuk, egyetlen nagy diszkként "látni"*: ez a **kötetesítés**.
- A *kötet*: logikai diszk, szervezhető rá fájlrendszer, van drivere.

Fájrendszer - ismertebb

- FAT (DOS), VFAT, exFAT, HPFS (OS/2),
- NTFS (NT),
- ext2/3/4 (Linux),
- XFS (SGI IRIX, Linux) - silicon graphics cég fejlesztette,
- Btrfs (B-tree file system) fájlrendszer – ez a Linux legújabb fájlrendszere,
- ZFS – Zettabyte File System (Solaris, stb.)

Linux fájlrendszerek – Raiser fájlrendszer

Hans Raiser és a Namesys fejlesztő csapat tervezte. *Előnyei:*

- 1. Jobb lemezterület kihasználás:** az összes adat egy kiegyensúlyozott B* fastruktúrába van szervezve, ezért jobban ki tudja használni a lemezterületet.
- 2. Jobb lemez hozzáférési teljesítmény:** kis fájlok esetén az *adatok* és a „*stat_data*” (inode) információ általában egymás mellett kerül tárolásra.
- 3. Gyorsabb helyreállítás összeomlás után:** ebbe a nyomkövető napló is támogatás nyújt.

Linux fájlrendszerek - ext2, ext3 és ext4

- **Ext2, Ext3 és Ext4** egy negyedik kiterjesztett fájlrendszer egy *naplózó fájlrendszer* Linux számára.
- **VFAT**, amely a FAT12 és a FAT16 bővítése volt, és a Windows NT 3.5-nél jelent meg.
- Az **XFS** egy nagy teljesítményű, [64 bites](#) naplózó fájlrendszer.
- A **/proc** fájlrendszer egy *látszólagos fájlrendszer*. A kernel hozza létre a memóriában.

Linux fájlrendszerek – Ext2

Az eredeti **Extended File System** 1992 áprilisában készült el és lett beépítve a Linux 0.96c-be.

Legfőbb erénye a **megbízhatóság**.

Előnye:

- *Gyorsabb* a többi fájlrendszerénél, mert az Ext2 nem tart karban naplót és kevesebb memóriát használ.

Hátránya:

- Tovább tart a fájlrendszer naplóadatainak ellenőrzése.

Linux fájlrendszerek – Ext3

Az Ext3 fájlrendszer egyszerűen ráépíthető egy Ext2 fájlrendszerre.

- *Egyszerű és nagyon megbízható* frissítés Ext2 fájlrendszerekről: a kód az Ext2-re épül, frissítés egyszerű.
- Megbízhatóság és teljesítmény.

Linux fájlrendszerek – BtrFS

„A BtrFS (B-tree File system) a legújabb Linuxos fájlrendszer.

Jellemzői:

- A *crc32c* támogatja *hibadetektáló és ellenőrzőkódok* és a fájlrendszer szintű *tömörítés*,
- *Partíció méret*: az *ext4* és az *NTFS* is kb. 1 EiB *támogat*, a Btrfs partíció akár 16 EiB is lehet.
- *Fájl méret*: *Ext4*-en a legnagyobb fájl 16 TiB lehet, Btrfs alatt akár a teljes partíció méretét is elfoglalhatja (16 EiB).

Linux fájlrendszerek – Btrfs

Továbbá támogatja:

- *Natív RAID* támogatás,
- *Beépített tartalom manager rendszert* tartalmaz (LVM),
- Támogatja a *kötetösszevonást*, azaz képes több merevlemezt, több partíciót egy kötegbe foglalni.
- *Deduplikálás* - ez azt jelenti, hogy a duplikált fájlok helyett képes egyet megtartani, ami a helytakarékossgot segíti.”

(Forrás: Linux Mint)

ZFS fájlrendszer

A *Sun Microsystems* fejlesztette, azonban amikor az *Oracle* ezt felvásárolta, akkor megalakult az *OpenZFS projekt*, ami tovább vitte a fejlesztést.

A *ZFS* tudja mindazt, amit a *Btrfs*, de ennek ellenére vannak különbségek.

Kötetméret:

- a *Btrfs* 16 exabájt (2 a 64. hatványon) *kötetméretet* támogat,
- *ZFS* 256 trillió yobibájt-ot. (2 a 128. hatványon) és nincs korlátja a fájlok számának.

ZFS fájlrendszer

- ZFS a *biztonságot főleg a CRC kezelésre alapozza*, a fájlsérüléseket automatikusan javítja (Btrfs a naplózásra).
- A COW-ot (Copy On Write) mindkettő tudja, de ZFS nagyobb redundanciával kezeli.

Hátránya: erőforrás igényes” (Forrás: Linux Mint)

Megj: COW-ot (Copy On Write): egy memória- és tároláskezelési technika, amely optimalizálja az erőforrások használatát azáltal, hogy csak akkor másol adatot, amikor az ténylegesen módosul.

Fájl szervezettség

Fájl fogalma? - valamilyen szempontból összetartozó adatok névvel ellátott együttese.

Fájl szervezettség (organisation): három általános struktúra lehetséges:

- *Byte-ok sora.*
- *Rekordok sora.*
- *Index-szekvenciális szervezettségű rekordok.*

Fájlrendszerek - Rekordok sora

Byte-ok sora (nincs szervezettség), - *nincs struktúráltság*, ill. a processzek struktúrálnak.

Rekordok sora (szekvenciális).

- A rekordok lehetnek *állandó*, vagy *változó hosszúságúak*, a rekordok a *blokkokba csoportosítva*.
- A *rekord struktúráltság* a partíción *rögzített* itt az *OS I/O alrendszere struktural*.

Fájlrendszerek

Index-szekvenciális szervezettségű rekordok:

- a rekordok *nem* feltétlenül *egyforma hosszúságúak*,
- a rekordokban a *mezők kulcsmezők (egyes)*, ezek *rendezettek*,
a rekordok gyors keresését teszik lehetővé.

Nem minden OS biztosítja.

Fájlrendszerek - A fájl elérése

A fájl elérése:

- *Szekvenciális (soros) elérés:* egy adatelem eléréséhez végig kell olvasni az előtte álló *bájtokat, rekordokat*.
- *Direkt (random) elérés:* közvetlenül elérhető bármelyik adatelem.

(pl. A UNIX *seek()* hívással lehetővé teszi.)

Fájlrendszerek - Fájl típusok

Fájl típusok:

- *közönséges fájl* (regular file): tovább is osztályozhatók (text fájl, binary fájl stb.)
- *jegyzékek* (directory-k): bejegyzéseket tartalmaznak további fájlokról.
- *speciális fájl*: eszköz azonosító, mailbox, FIFO jellegű fájl.

Fájrendszeresek - Fájrendszer megvalósítások

Partíciókra szervezhetünk fájlrendszer

Három megoldandó kérdés:

1. Adott fájlhoz a *blokkok hozzárendelése*.

(Keresni, hozzáfűzni, elengedni blokkokat),

2. Az eszközön a *szabad blokkok menedzselése* (Keresni szabad blokkokat, elengedni (szabadra tenni) blokkokat.).

3. A fájl *attribútumainak tárolása*, ill. a *jegyzékstruktúra kialakítása*.

Fájrendszer

Blokk allokációs módszerek

- folytonos allokáció,
- láncolt listás allokáció,
- láncolt listás allokáció index táblával,
- i-node módszer.

HPFS (High Performance File System - nagy teljesítményű FS)

Fájlrendszerek - Folytonos blokk allokáció

Előnye: Egyszerű a nyilvántartás, a blokkok visszakeresése, teljesítményre optimalizált allokáció.

Hátránya: A változó partíciós memória kezelésnél tanultak érvényesek:

- Elhelyezés lyukakba (*first fit, next fit, worst fit, best fit*).
- Nagy a töredezettség, a kihasználatatlan terület.
- Problémát jelenthet a *hozzáfűzés (append)* meglévő fájlhoz.

Fájlrendszerek - Láncolt listás blokk allokáció

Láncolt listás blokk allokáció

- Elhelyezés *több darabban*, nem *feltétlenül folytonosan*.
- Az adott blokkban *mutató a következő blokkra*.
- Az *utolsó blokkban a mutató a véget jelez(het)i*.

Fájlrendszerek - Láncolt listás blokk allokáció

Előnye:

- Egyszerű, nincs fragmentáció, könnyű a hozzáfűzés stb.

Hátránya:

- *Hosszú fájlok* végének eléréséhez végig kell menni a láncolt listán.
- A *mutató* a blokkokon “elvesz” területet a fájl adataitól.

Keverednek a *fájl adatok* és a *nyilvántartási adatok*.

Fájlrendszerek - Láncolt listás blokk allokáció indextáblával (FAT)

Láncolt listás blokk allokáció indextáblával (FAT)

*A fájl **blokkokra osztva** helyezzük el a partíció blokkjaiba, **nem feltétlenül folytonosan**.*

*Az **első blokk címe** a jegyzékben (mint attrib.)*

*A **mutató nem a blokkba kerül** - hanem **indextáblába** kerül.*

Fájlrendszerek - Indextábla

Indextábla: *egy fix helyen* található a partíción, egy *bejegyzése egy-az-egy* megfeleltetésben a *blokkokkal* (i-edik bejegyzéshez az i-edik blokk tartozik).

A bejegyzés az indextáblában kettős jelentésű:

- a bejegyzés *index-száma* jelzi, az ugyanolyan számú *blokk a fájl blokkja*.
- maga a *bejegyzés a soron következő blokk* (egyben index is), vagy a *lista vége*.

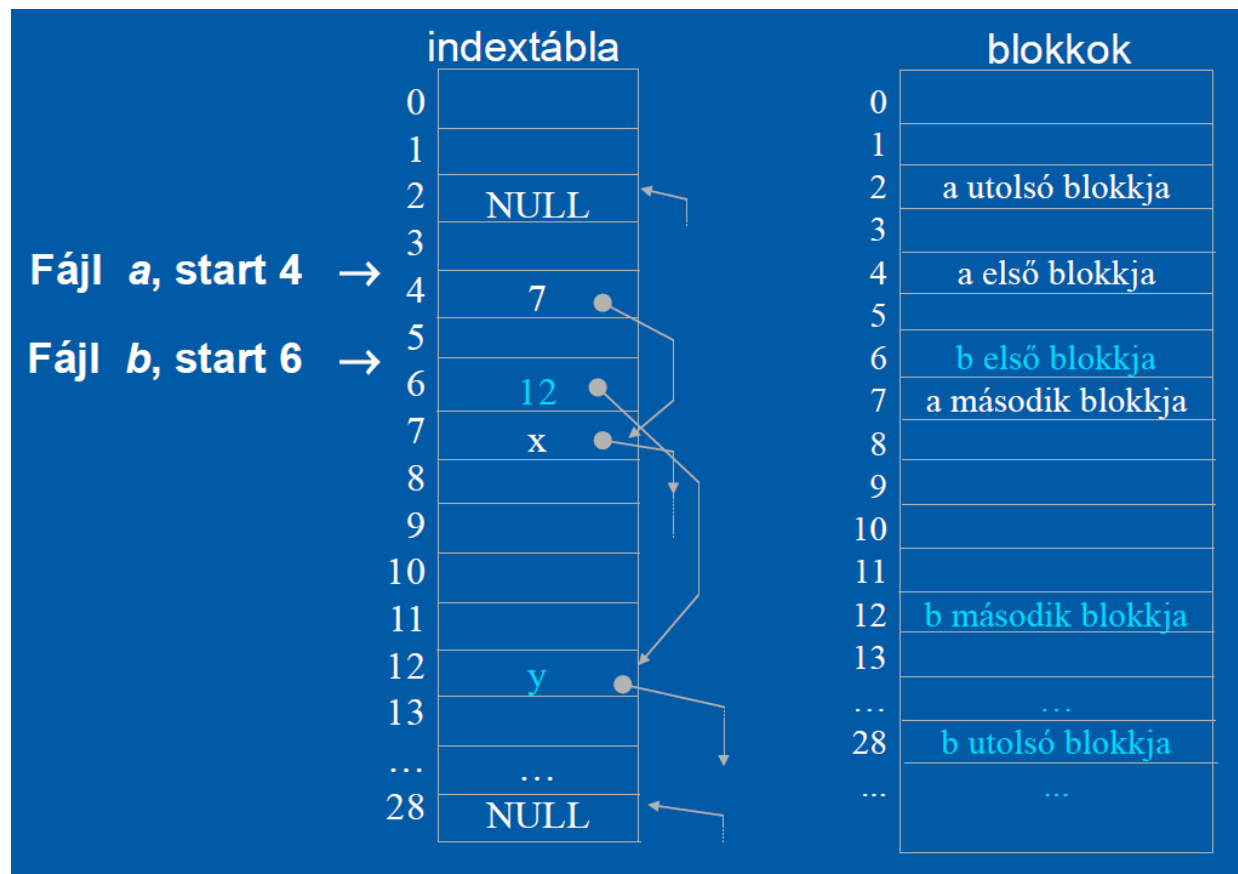
Fájlrendszerek - Indextábla

- Tulajdonképp láncolt lista, de a *mutatók ki* vannak emelve az *indextáblába*.
- Nem zárja ki az optimalizált allokációs lehetőségeket.

Előnyei:

- Közvetlen hozzáférés.

Fájrendszeres – Indextáblás allokáció



Forrás: VadászD

Fájrendszer - Láncolt listás blokk allokáció indextáblával (FAT)

Előnyei:

- Nincs fragmentáció.
- Egy blokk megkereséséhez nem kell a megelőző blokkokat kiolvasni, *csak az indextáblán kell végig követni.*

Fájrendszer - Láncolt listás blokk allokáció indextáblával (FAT)

Hátrányai:

- Nagy *partíciókon, nagy index tábla.*
- *Sérülékenység* (mi van ha elvesz az index tábla?)

Fájrendszer- Láncolt listás blokk allokáció indextáblával (FAT) – nagy index tábla

Láncolt listás blokk allokáció indextáblával (FAT)

Nagy index tábla probléma megoldása:

Cluster-ek: több blokk együtt összevonás, csak *cluster-eket* lehet foglalni, clusterenként bejegyzés az indextáblában.

Hátránya: nő a kihasználhatatlan terület (minimum egy cluster foglalható meg a kisebbnek is.)

Fájrendszerrek - Láncolt listás blokk allokáció indextáblával (FAT)

Sérülékenység probléma megoldása:

Dupla index tábla.

Hátránya: minden változást két helyen kell átvezetni.

Fájrendszeres - I-node-s blokk allokáció

I-node-s blokk allokáció (pl. UNIX fájlrendszer)

Az i-node szám (*index node*) a file-t azonosító adat:

- 1 file-hoz 1 i-node szám,
- 2 byte = 65.536 db file.

Elhelyezés *blokkokra osztva* a partíció tetszőleges blokkjaiba.

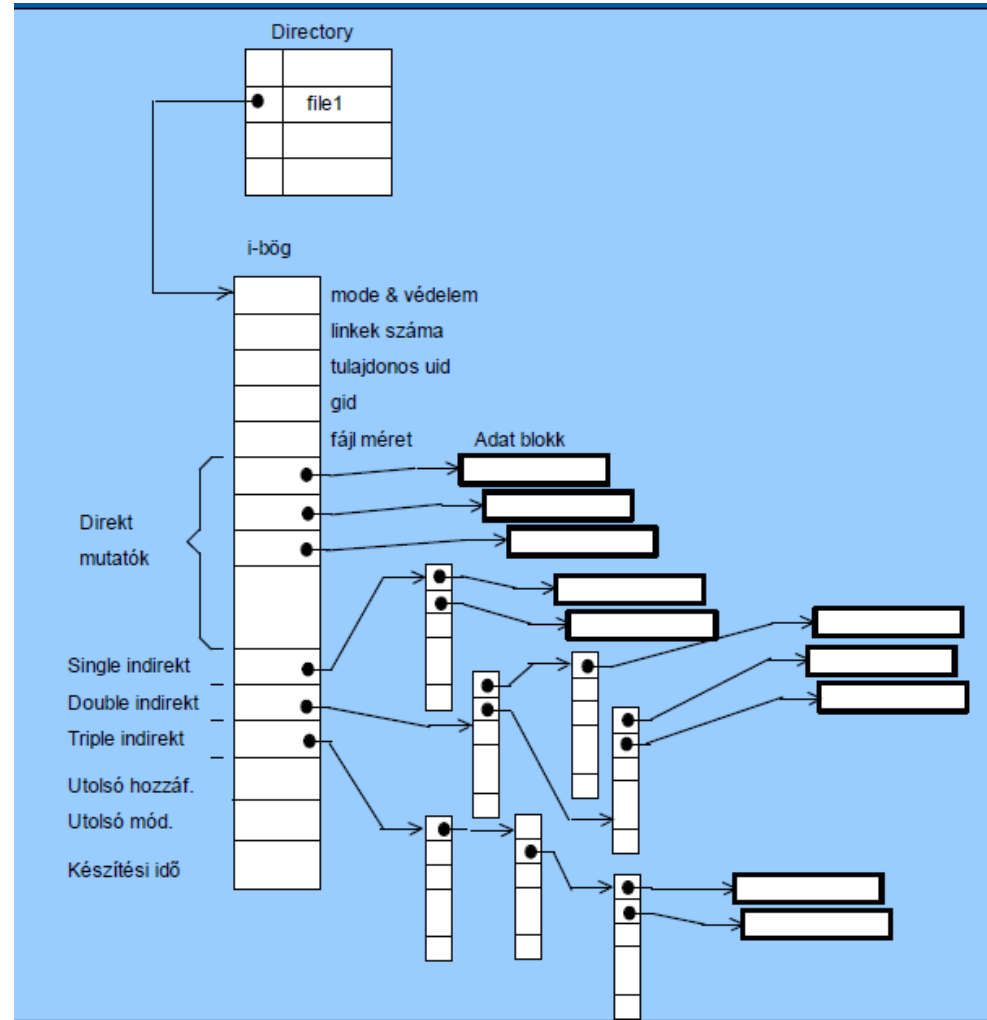
Minden fájlhoz tartozik egy táblázat, amely mutatja, hogy mely blokkja, mely blokkba van elhelyezve.

Fájrendszeres - I-node-s blokk allokáció

A táblázat többszintű pl.

- *0-9 sora*: tartalmazza a *fájl 0-9 blokkjának címet* (direkt).
- *10-12 sora*: *indirekt blokkmutatókat* tartalmaz, azaz egy-egy 10 soros táblázatra mutatnak, amely tartalmazza a következő 10 blokk címet (10-39).
- *13-15 sora*: *kétszeresen indirekt blokkmutatókat* tartalmaz (40-339).

A UNIX i-node szerkezete



Forrás: Vadász

Fájrendszeres - I-node-s blokk allokáció

I-node-os blokk allokáció

Egy *i-node-ban* található: a *fájl adatai*, a *blokkcímek fő táblázata*, és *egyéb adatok* (dátumok, jogosultságok),.

A *jegyzékben* csak a *fájl neve* és *i-node száma* van.

Az *i-node-ok* a partíció egy elkülönített területen vannak elhelyezve.

Fájlrendszerek - I-node-os blokk allokáció

Előnyök:

- Nincs fragmentáció.
- Egy *blokk megtalálásához kevesebbet* kell olvasni, mint bármelyik megelőző módszernél.
- A fájl elhelyezési információk *egy helyről elérhetők* (nem pedig jegyzék + indextábla).

Hátránya:

- A szabad blokkok külön nyilvántartás igényelnek (index tábla egyben szabad blokk nyilvántartás is).

Fájrendszeresek - Szabad blokkok kezelése

2. Szabad blokkok kezelése

- Bittérképes (bitmap)
- Láncolt listás (linked list)

Fájrendszeres - Bittérképes

Szabad blokk nyilvántartás: Bittérképes

- Kötött helyen a bit-térkép - lemez kijelölt helyén van.
- Egy-az-egy megfeleltetés *egy bit* és a *blokkok* (clusterek) között.
- Ha egy bit bebillentett: azt jelenti, a hozzá tartozó blokk foglalt.
- Ilyen pl. a VAX/VMS, HPFS, NTFS, Linux ext2/ext3.

Fájlrendszerek - Láncolt listás

Szabad blokkok kezelése: Láncolt listás

- Egy blokk *mutatókat tartalmaz szabad blokkokra*.
- Egy *pointere* viszont további olyan blokkra mutat, ami szinten szabad blokkok mutatóit tartalmazza.
- Viszonylag kevés blokk lánc tartalmazhatja az összes szabad blokkot.

Fájlrendszerek - Láncolt listás

- Egyszerű.
- Nehezebb módosíthatóság.
- A lista elejéről a következő szabad blokk.
- Az első szabad blokk címének tárolása, arra felfűzve a többi.
- A blokk területéből vesszük le a cím területét.
- Nem hatékony, lassú (sok lemezművelet).

Fájlrendszerek - Jegyzék struktúra

Jegyzék struktúra - *a jegyzék is fájl.*

Tartalma: Kötött vagy változó hosszú lehet.

Tartalmazhat: *bejegyzések neveit, i-node számot vagy egyéb adatokat* (kezdő címet, hosszat, attribútumokat, stb.).

Gyakori a keresés a jegyzék fájlban belül, ezért megfelelő szerkezet (hash tábla, b-fa).

Fájrendszer - Általános UNIX fájlrendszer

Itt minden fájl: az *eszközök is*, a fájlrendszer *blokkos eszközre* (partícióra, kötetre) szervezett.

- A partíciónak van egy *superblokkja*, itt információk magáról a fájlrendszeréről.
- Az allokációhoz *i-node-os módszert* használ.
- A szabad blokk menedzseléshez *láncolt listát* használ, a szabad blokkokat felhasználva. *Kiindulás a superblokkból.*

Fájlrendszerek - A UNIX eszközök szimbólikus nevei

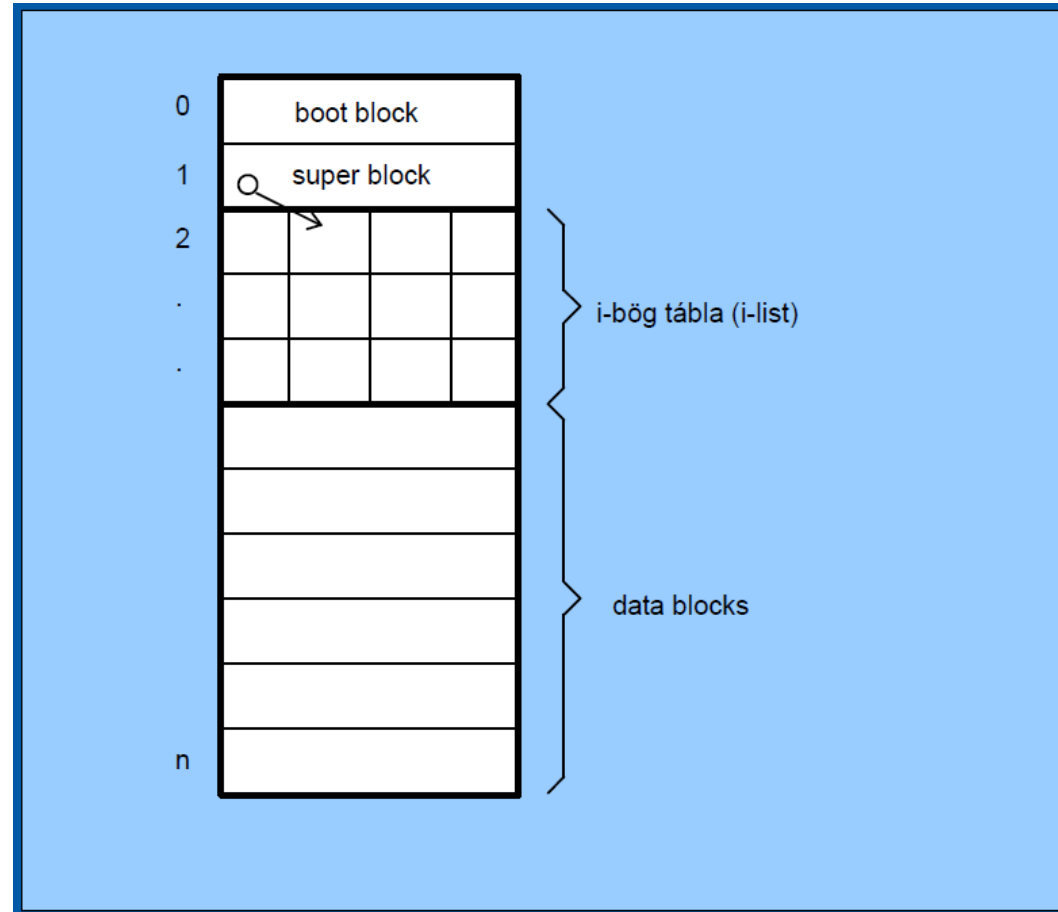
Az eszközök: speciális fájlok, a /dev/ jegyzékben, aljegyzékeiben ...

Tartalmuk:

- **major device number:** a controllert azonosítja ,
- **minor device number:** a tényleges eszközt (pl. logikai diszket), és a device driver-t azonosítja.

Egy UNIX partíció szerkezete

File-rendszer a lemezen:



Forrás: Vadász

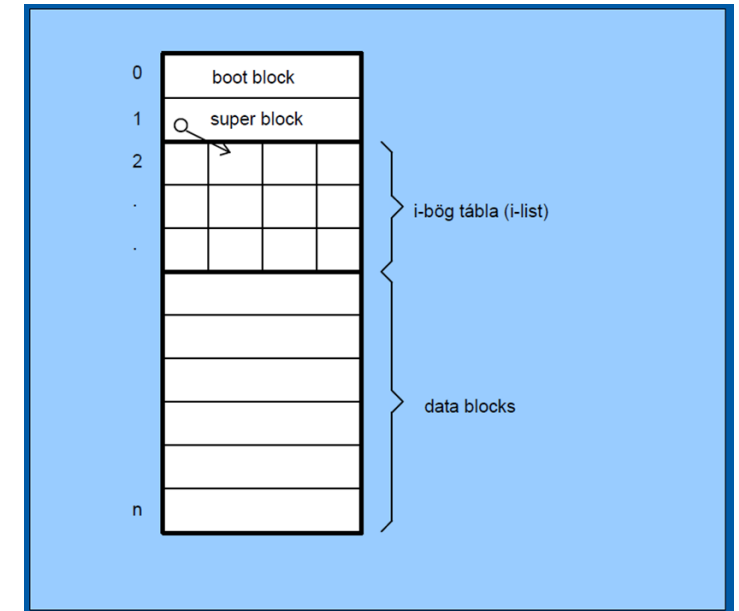
Egy UNIX partíció szerkezete – boot block

Boot block:

- *a rendszer indításához szükséges információkat tartalmazza,*
- *bár csak 1 file-rendszernek szükséges, mindegyik tartalmazza, legfeljebb üres.*

UNIX fájlrendszer – szuperblokk tartalma

- a fájlrendszer mérete,
- a szabad blokkok száma,
- a szabad blokkok listája + index a listán,
- az i-lista mérete,
- a szabad i-bögek (*i-node*) száma,
- a szabad i-bögek listája + index a listához,
- jelző, hogy történt-e módosítás a szuperblokkban.



UNIX fájlrendszer – i-node, datablocks

i-node list:

- az i-node-ok *nyilvántartásának a helye,*
- a file-rendszerben maximálisan létrehozható file-ok számát határozza meg (super user által).

Datablocks:

- az *adatok fizikai tárolási helyei.*

A UNIX file-rendszer felhasználói interface-e

A UNIX file-rendszer *felhasználói felületének lehetőségei*:

- standard input/output átirányítás,
- mount: két file-rendszer logikai összekapcsolása,
- link-ek létrehozása:
 - hard link,
 - szimbolikus link (csak FFS-ben).

Fájlrendszerek - UNIX fájlrendszer

Link: célja a már meglévő fájlra más névvel is hivatkozhasunk.

Hard link:

- új dir-bejegyzés készül, a már létező fájl i-node-jára hivatkozva!
- Csak ugyanazon a fájlrendszeren!
- Törléskor a linkek száma csökken!

Fájlrendszerek - UNIX fájlrendszer

Soft link (symbolic link):

- új fájl készül (új i-böggel), de az eredeti file-ra mutat (abszolút útvonal)”[1][2][3][4][5][6]

Felhasznált irodalom

1. Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

2. Vincze Dávid: Operációs rendszerek, ME, 2019.

3. Operációs rendszerek

<https://dcs.uni-pannon.hu/oktatas/tantargyak/operacios-rendszerek>

4. Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

Irodalom

5. Operációs rendszerek

<https://vik.wiki.hu>

6. Linux Mint

https://linuxmint.hu/sites/default/files/fajlok/linux_mint_kezikonyv.pdf